

CSC212 Project

Data Structures

E-Commerce Inventory &

Order Management System

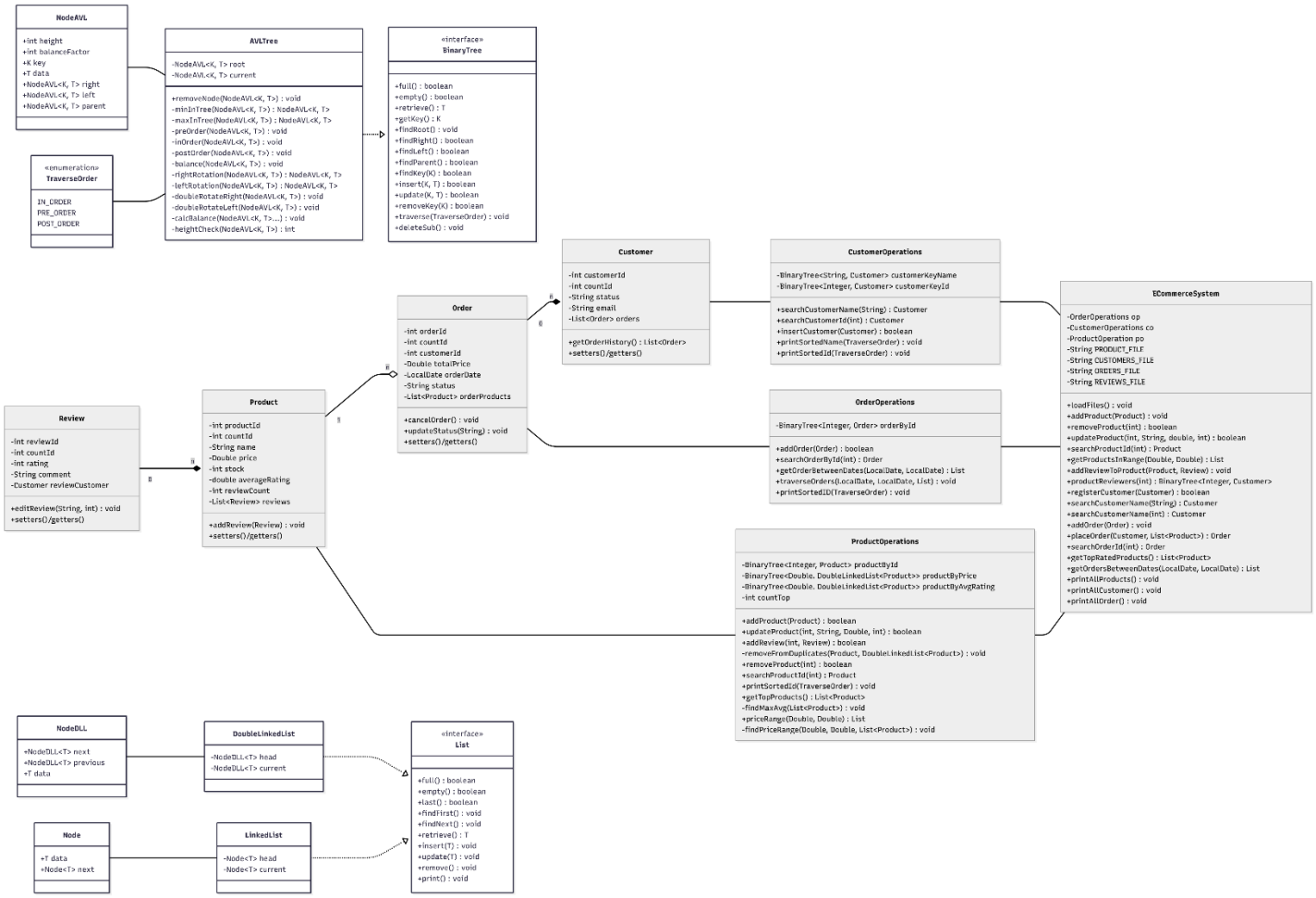
PHASE 2

Yazan Almuzaini 445103365

Abdulaziz Alnahedh 445102328

Sultan Almandeel 445100971

Class Diagram:



Phase 1 / Phase 2 Search Comparison:

Methods	Phase 1		Phase 2	
	Time Complexity	Space Complexity	Time Complexity	Space Complexity
Search Product by Id	O(n)	O(1)	O(log n)	O(1)
Search Customer/Product by Name	O(n)	O(1)	O(log n)	O(1)
Search Order by Id	O(n)	O(1)	O(log n)	O(1)

All Required Methods, with Big O analysis:

Two structures were needed, an AVL Tree and a Double/Single Linked List

Class: AVLTree<K extends Comparable<K>, T>

Method: balance(NodeAVL<K, T>)



```java
private void balance(NodeAVL<K, T> node) { // recursive
        calcBalance(node);

        if (node.balanceFactor == 2) {
            if (node.right.balanceFactor >= 0) {
                // rotate right
                rightRotation(node);
            } else {
                // double rotate
                doubleRotateRight(node);
            }
        }

        else if (node.balanceFactor == -2) {

            if (node.left.balanceFactor <= 0) {
                // rotate left
                leftRotation(node);
            } else {

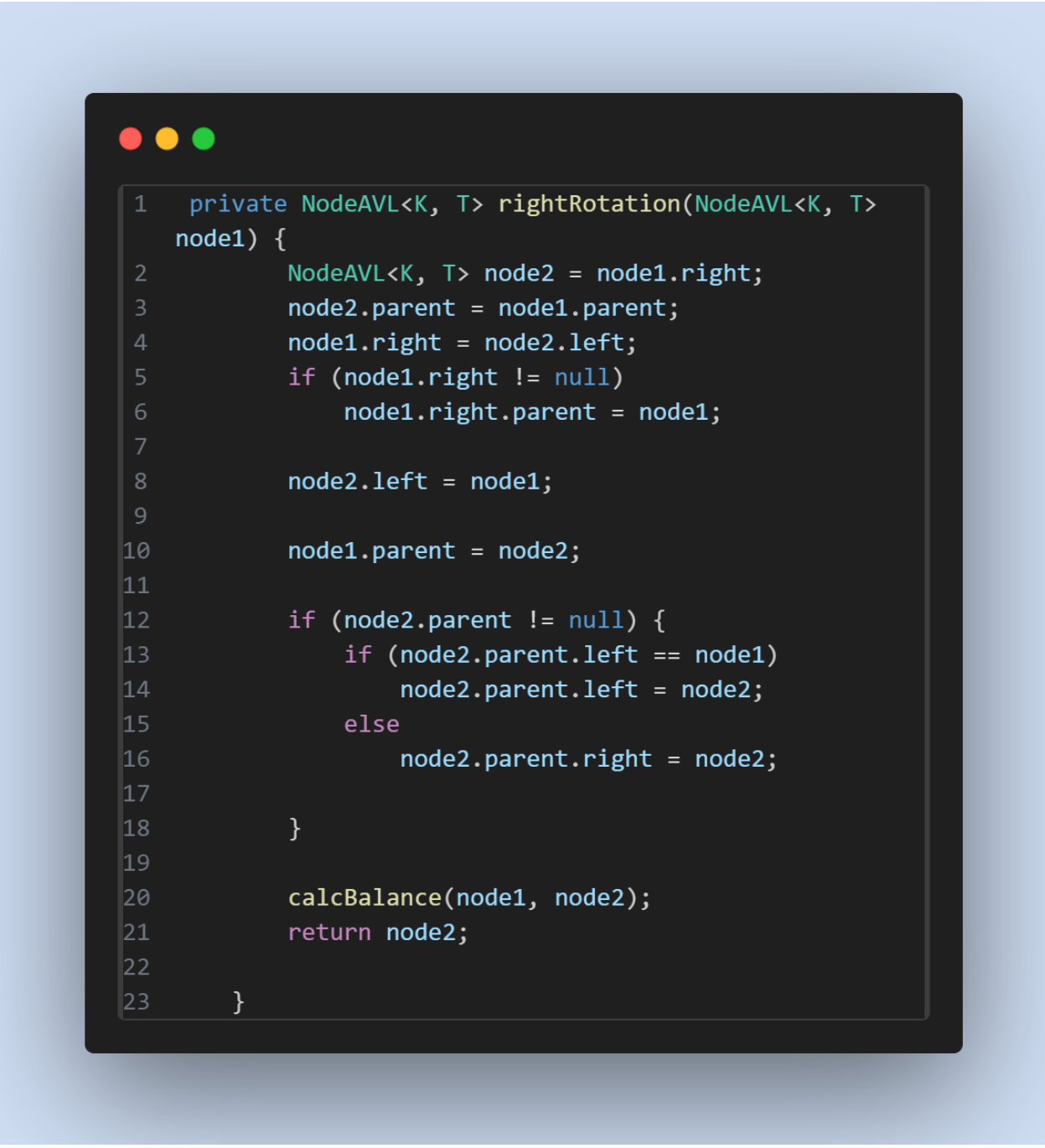
                // double rotate
                doubleRotateLeft(node);
            }

        }
        // call method again for node.parent
        if (node.parent != null)
            balance(node.parent);
        else
            root = node;
    }
```

Description: Balances the sub-tree at the specified node.

- Time complexity: O(log n)
- Space Complexity: O(1)

Method: rightRotation



```
1   private NodeAVL<K, T> rightRotation(NodeAVL<K, T>
    node1) {
2           NodeAVL<K, T> node2 = node1.right;
3           node2.parent = node1.parent;
4           node1.right = node2.left;
5           if (node1.right != null)
6               node1.right.parent = node1;
7
8           node2.left = node1;
9
10          node1.parent = node2;
11
12          if (node2.parent != null) {
13              if (node2.parent.left == node1)
14                  node2.parent.left = node2;
15              else
16                  node2.parent.right = node2;
17
18          }
19
20          calcBalance(node1, node2);
21          return node2;
22
23      }
```

Description: Rotates the right node to the top.

- Time complexity: O(1)
- Space Complexity: O(1)

Method: leftRotation

```java
private NodeAVL<K, T> leftRotation(NodeAVL<K, T>
node1) {
        NodeAVL<K, T> node2 = node1.left;
        node2.parent = node1.parent;

        node1.left = node2.right;
        if (node1.left != null)
            node1.left.parent = node1;

        node2.right = node1;
        node1.parent = node2;

        if (node2.parent != null) {
            if (node2.parent.left == node1)
                node2.parent.left = node2;
            else
                node2.parent.right = node2;

        }

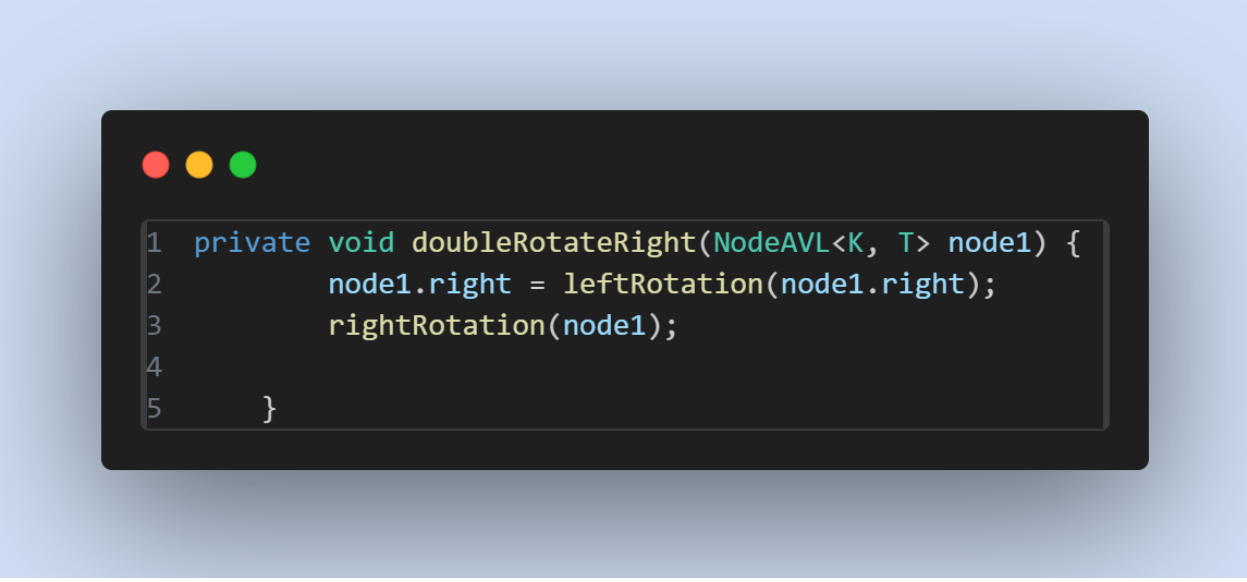
        calcBalance(node1, node2);
        return node2;

    }
```

Description: Rotates the left node to the top.

- Time complexity: O(1)
- Space Complexity: O(1)

Method: doubleRotateRight

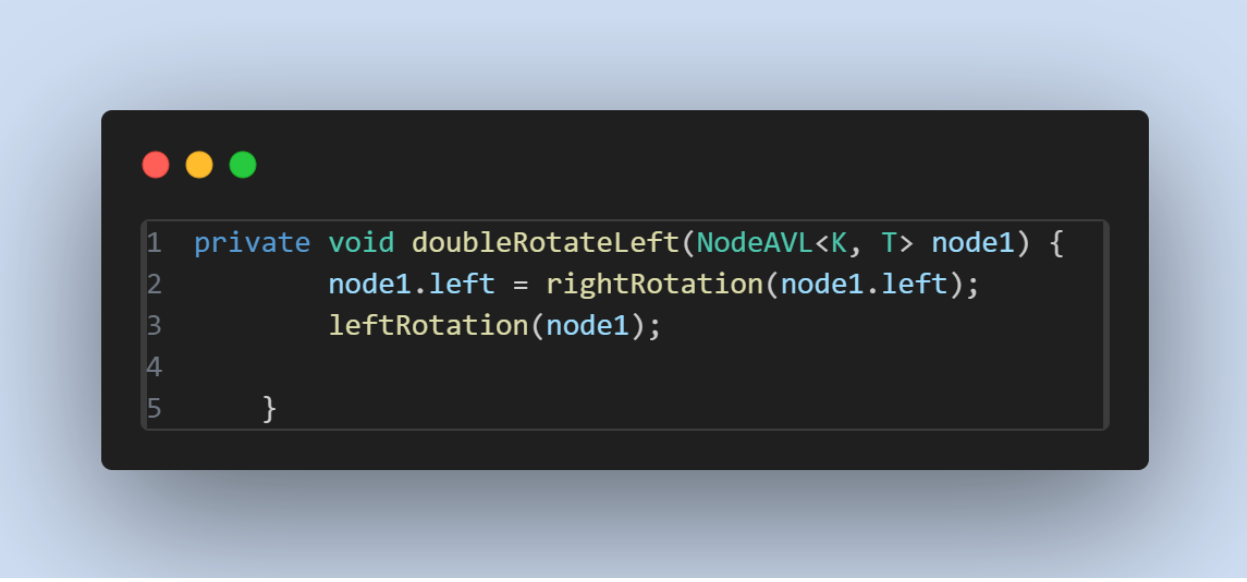


```
1  private void doubleRotateRight(NodeAVL<K, T> node1) {
2          node1.right = leftRotation(node1.right);
3          rightRotation(node1);
4
5      }
```

Description: This is performed when a node is right heavy, but the right child is left heavy.

- Time complexity: O(1)
- Space Complexity: O(1)

Method:doubleRotateLeft

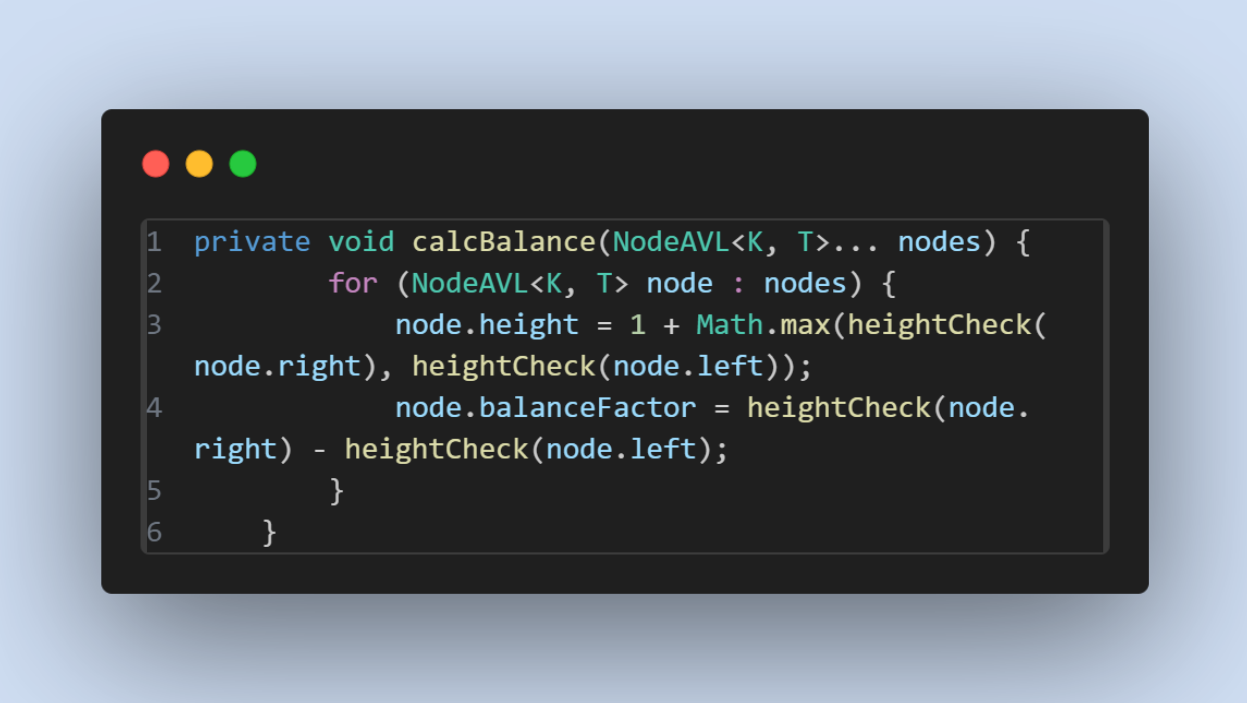


```
1  private void doubleRotateLeft(NodeAVL<K, T> node1) {
2          node1.left = rightRotation(node1.left);
3          leftRotation(node1);
4
5      }
```

Description: This is performed when a node is left heavy but the left child is right heavy.

- Time complexity: O(1)
- Space Complexity: O(1)

Method: calcBalance

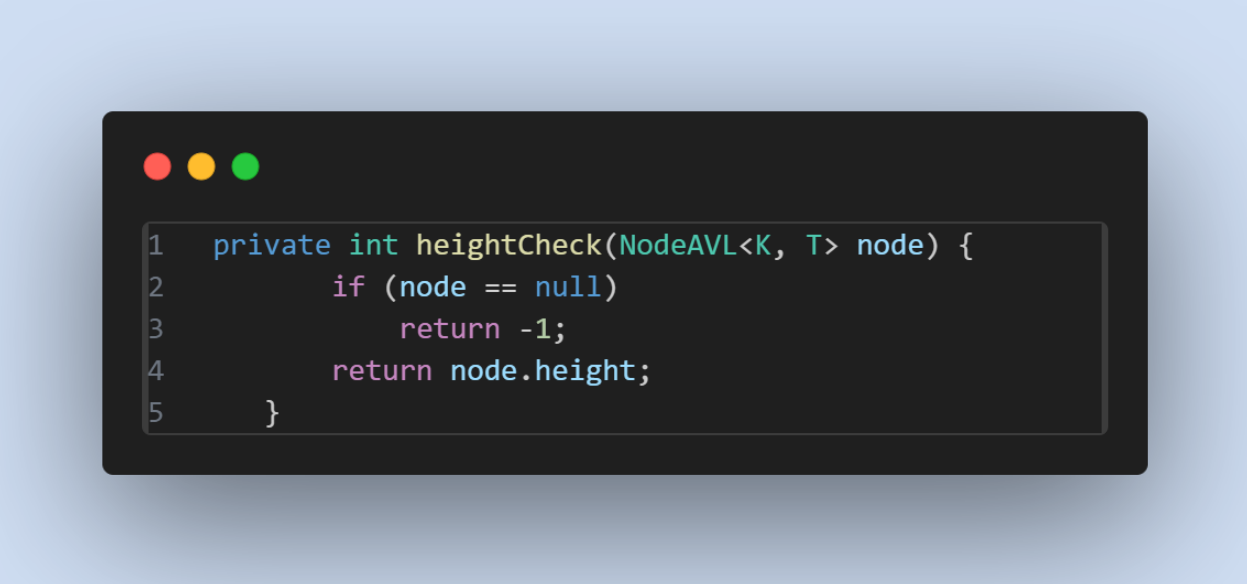


```
1  private void calcBalance(NodeAVL<K, T>... nodes) {
2          for (NodeAVL<K, T> node : nodes) {
3              node.height = 1 + Math.max(heightCheck(
   node.right), heightCheck(node.left));
4              node.balanceFactor = heightCheck(node.
   right) - heightCheck(node.left);
5          }
6      }
```

Description: Calculates the balance factor by subtracting the height of the right child from the height of the left child.

- Time complexity: O(number of nodes this is called on)
- Space Complexity: O(1)

Method: heightCheck



```java
private int heightCheck(NodeAVL<K, T> node) {
        if (node == null)
            return -1;
        return node.height;
    }
```

Description: To avoid null values being returned.

- Time complexity: O(1)
- Space Complexity: O(1)

Class: ProductOperations

Method: addProduct



```java
public boolean addProduct(Product p) {
        if (productById.insert(p.getProductId(), p)) {
            productByPrice.insert(p.getPrice(), new DoubleLinkedList<Product>());
            productByPrice.retrieve().insert(p);

            productByAvgRating.insert(p.getAverageRating(), new DoubleLinkedList<Product>());
            productByAvgRating.retrieve().insert(p);
            return true;
        } else
            return false;
    }

```

Description: Performs 3 different inserts for each tree.

- Time Complexity: O(3 * log(n)) = O(log n)
- Space Complex: O(1)

Method: updateProduct



```java
public boolean updateProduct(int id, String name, Double price, int stock) {
        if (productById.findKey(id)) {
            Product p = productById.retrieve();
            p.setName(name);
            p.setStock(stock);

            productByPrice.findKey(p.getPrice());

            DoubleLinkedList<Product> products = productByPrice.retrieve();

            removeFromDuplicates(p, products);

            if (products.empty()) {
                productByPrice.removeKey(p.getPrice());
            }

            p.setPrice(price);

            productByPrice.insert(price, new DoubleLinkedList<>());
            productByPrice.retrieve().insert(p);

            return true;
        }

        return false;
    }
```

Description: Update three different trees, two of them may contain multiple products within one node (When price/rating is the same).

- Time complexity: O(log(n) + k), ‘k’ is number of duplicate items, mostly low.
- Space Complexity: O(1)

Method: addReview



```java
public boolean addReview(int id, Review r) {
        if (productById.findKey(id)) {
            Product p = productById.retrieve();

            Double avg = p.getAverageRating();

            productByAvgRating.findKey(avg);

            DoubleLinkedList<Product> products = productByAvgRating.retrieve();

            removeFromDuplicates(p, products);

            if (products.empty()) {
                productByAvgRating.removeKey(avg);
            }

            p.addReview(r);

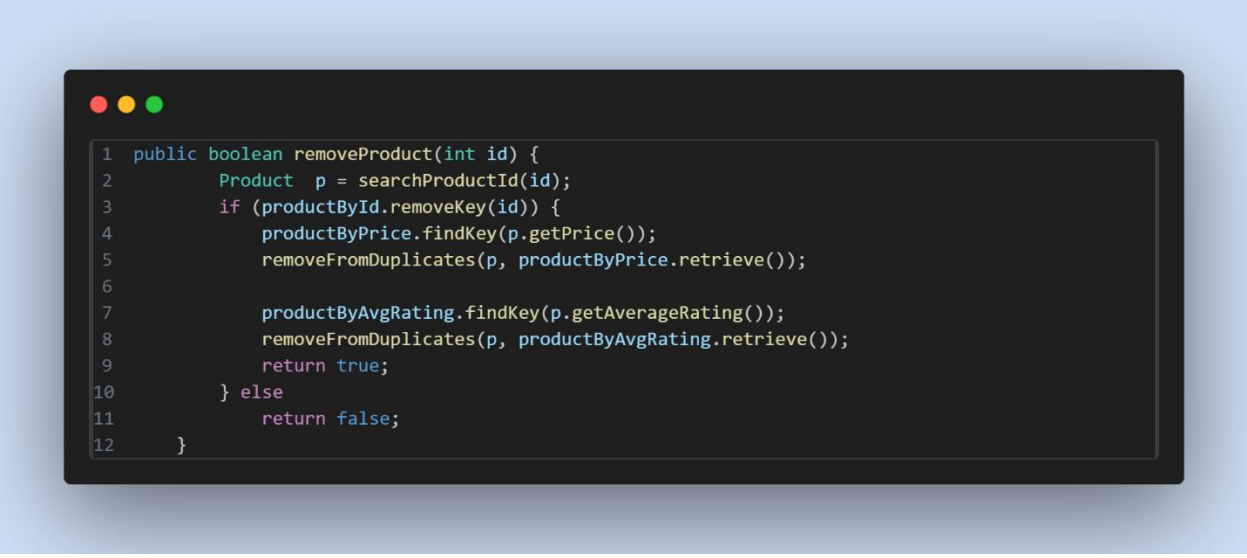
            productByAvgRating.insert(p.getAverageRating(), new DoubleLinkedList<>());
            productByAvgRating.retrieve().insert(p);

            return true;
        }
        return false;
    }
```

Description: inserts a review to the product's "reviews" list, and updates the average rating tree.

- Time complexity: O(log(n) + k), 'k' is number of duplicate items, mostly low.
- Space Complexity: O(1)

Method: removeProduct



```java
public boolean removeProduct(int id) {
        Product  p = searchProductId(id);
        if (productById.removeKey(id)) {
            productByPrice.findKey(p.getPrice());
            removeFromDuplicates(p, productByPrice.retrieve());

            productByAvgRating.findKey(p.getAverageRating());
            removeFromDuplicates(p, productByAvgRating.retrieve());
            return true;
        } else
            return false;
    }
```

Description: Removes a product from three different trees simultaneously.

- Time complexity: O(log(n) + k), ‘k’ is number of duplicate items, mostly low.
- Space Complexity: O(1)

Method: searchProductId



```java
public Product searchProductId(int productId) {
        Product p = null;
        if (productById.findKey(productId))
            p = productById.retrieve();
        return p;
    }
```

Description: Searches the tree keyed by product Id.

- Time complexity: O(log n), was O(n)
- Space Complexity: O(1)

Method: getTopProducts



```java
public List<Product> getTopProducts() {
        List<Product> top = new LinkedList<>();

        productByAvgRating.findRoot();

        countTop = 0;
        findMaxAvg(top);

        return top;
    }

    private void findMaxAvg(List<Product> result) {
        if (countTop >= 3)
            return;

        if (productByAvgRating.findRight()) {
            findMaxAvg(result);
            productByAvgRating.findParent();
        }

        if (countTop >= 3)
            return;

        DoubleLinkedList<Product> products = productByAvgRating.retrieve();

        products.findFirst();
        while (true) {
            result.insert(products.retrieve());
            if (++countTop >= 3)
                return;
            if (products.last())
                break;
            products.findNext();
        }

        if (countTop < 3) {
            if (productByAvgRating.findLeft()) {
                findMaxAvg(result);
                productByAvgRating.findParent();
            }
        }
    }
```

Description : Performs a reverse “In Order” traversal but stops after collecting three nodes.

- Time complexity: O(log n)
- Space Complexity: O(log n), but the average case O(1)

Method: priceRange



```java
public List<Product> priceRange(Double minPrice, Double maxPrice) {
        List<Product> productsInRange = new LinkedList<>();

        productByPrice.findRoot();

        findPriceRange(minPrice, maxPrice, productsInRange);

        return productsInRange;

    }

    private void findPriceRange(Double minPrice, Double maxPrice, List<Product> result) {
        Double key = productByPrice.getKey();

        if (key >= minPrice && key <= maxPrice) {

            DoubleLinkedList<Product> products = productByPrice.retrieve();

            products.findFirst();
            while (true) {
                result.insert(products.retrieve());
                if (products.last())
                    break;
                products.findNext();
            }

            if (productByPrice.findLeft()) {
                findPriceRange(minPrice, maxPrice, result);
                productByPrice.findParent();
            }

            if (productByPrice.findRight()) {
                findPriceRange(minPrice, maxPrice, result);
                productByPrice.findParent();
            }

        }

        if (key < minPrice) {
            if (productByPrice.findRight()) {
                findPriceRange(minPrice, maxPrice, result);
                productByPrice.findParent();
            }

        }

        if (key > maxPrice) {
            if (productByPrice.findLeft()) {
                findPriceRange(minPrice, maxPrice, result);
                productByPrice.findParent();
            }
        }

    }
```

Description : Traverses the tree using range-based logic, as in going left until key<min and stopping going right until key>max.

- Time complexity: O(log n)
- Space Complexity: O(log n)

Class: CustomerOperations

Method: insertCustomer

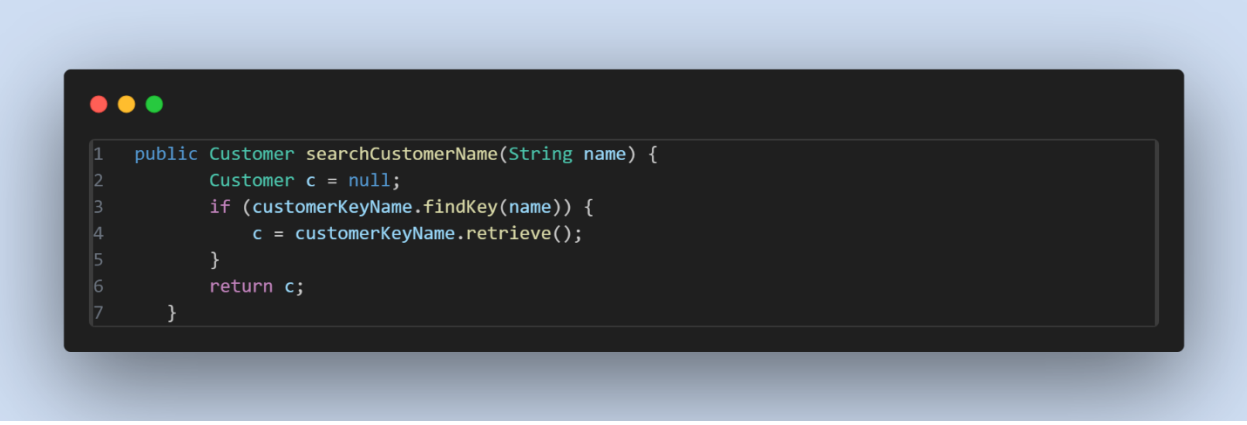


```java
public boolean insertCustomer(Customer c) {
        if (customerKeyId.insert(c.getCustomerId(), c)) {
            if (customerKeyName.insert(c.getName(), c)) {
                return true;
            } else {
                customerKeyId.removeKey(c.getCustomerId());
            }
        }
        return false;
    }
```

Description: Inserts a customer to two different trees.

- Time complexity: O(log n)
- Space Complexity: O(1)

Method: searchCustomerName



```java
public Customer searchCustomerName(String name) {
        Customer c = null;
        if (customerKeyName.findKey(name)) {
            c = customerKeyName.retrieve();
        }
        return c;
    }
```

Description: Searches the tree of customers keyed by name.

- Time complexity: O(log n)
- Space Complexity: O(1)

Method: searchCustomerId



```
public Customer searchCustomerId(int id) {
        Customer c = null;
        if (customerKeyId.findKey(id)) {
            c = customerKeyId.retrieve();
        }
        return c;
    }

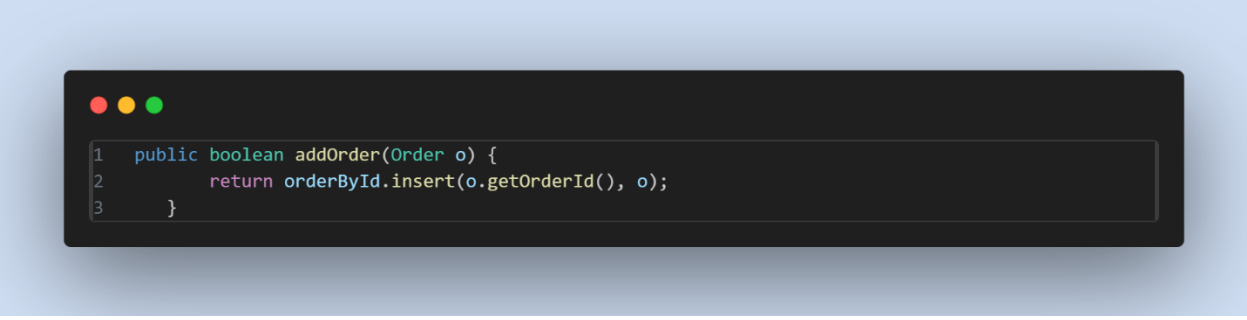
```

Description: Searches tree keyed by customer Id.

- Time complexity: O(log n), was O(n)
- Space Complexity: O(1)

Class: OrderOperations

Method: addOrder



```java
public boolean addOrder(Order o) {
        return orderById.insert(o.getOrderId(), o);
    }
```

Description: Inserts order to one tree.

- Time Complexity: O(log n)
- Space Complexity: O(1)

Method: searchOrderById



```java
public Order searchOrderById(int id) {
        Order o = null;
        if (orderById.findKey(id)) {
            o = orderById.retrieve();
        }
        return o;
    }
```

Description: Searches tree keyed by order Id.

- Time Complexity: O(log n), was O(n)
- Space Complexity: O(1)

Method: getOrderBetweenDates



```java
public List<Order> getOrdersBetweenDates(LocalDate start, LocalDate end) {
        List<Order> orders = new LinkedList<>();
        orderById.findRoot();
        traverseOrders(start, end, orders);
        return orders;
    }

    public void traverseOrders(LocalDate start, LocalDate end, List<Order> result) {
        if (orderById.findLeft()) {
            traverseOrders(start, end, result);
            orderById.findParent();
        }
        Order o = orderById.retrieve();
        if (o.getOrderDate().isAfter(start) && o.getOrderDate().isBefore(end)) {
            result.insert(o);
        }

        if (orderById.findRight()) {
            traverseOrders(start, end, result);
            orderById.findParent();
        }
    }
```

Description: Traverses the tree keyed by Id using “In Order” traversal.

- Time Complexity: O(n), could be done at O(log n), but an “In order” traversal was required
- Space Complexity: O(log n)

Class: ECommerceSystem

Method: productReviewer (result is sorted by customer Id)



```java
public BinaryTree<Integer, Customer> productReviewers(int id) {
        BinaryTree<Integer, Customer> customers = new AVLTree<>();
        Product p = po.searchProductId(id);
        if (p != null) {
            List<Review> reviewsOfProduct = p.getReviews();
            if (!reviewsOfProduct.empty()) {
                reviewsOfProduct.findFirst();
                while (true) {
                    Customer reviewer = reviewsOfProduct.retrieve().getReviewCustomer();
                    customers.insert(reviewer.getCustomerId(), reviewer);
                    if (reviewsOfProduct.last())
                        break;
                    reviewsOfProduct.findNext();
                }
            }

        }
        return customers;

    }
```

Description: Searches for the product in the Id tree, accesses reviews list, then iterate over list, inserting customer to an AVL tree with each iteration.

- Time Complexity: O(log n + r * log r), where ‘r’ is the number of reviews of a product.
- Space Complexity: O(r)